

Prompt Engineering Impact on Large Language Model Reasoning

Palak Singh

Department of Computer Science

Axis Institute of Technology & Management, Uttar Pradesh, India

palak0001singh@gmail.com

Abstract—While Large Language Models (LLMs) excel in pattern recognition, executing reliable multi-step logical inference remains a bottleneck, particularly for small-scale architectures. They have achieved impressive performance across various tasks; however, their ability to perform reliable multi-step reasoning remains a significant challenge. Recent research suggests that prompt engineering plays a crucial role in reasoning capabilities from these models. This paper investigates the impact of different prompt engineering strategies on LLM reasoning performance, with particular focus on mathematical and logical reasoning tasks. We evaluate zero-shot, few-shot, and chain-of-thought prompting approaches using a controlled experimental setup involving TinyLLaMA-1.1B-Chat-v1.0, a lightweight open-source language model executed locally via LM Studio. The performance of each strategy is assessed based on answer correctness, logical consistency, and reasoning clarity. Few-shot prompting similarly demonstrates limited gains when the model lacks sufficient representational depth. An in-depth error analysis reveals recurring patterns of incorrect arithmetic transformations, unnecessary floating-point conversion, inconsistent operator precedence handling, and failure to apply standard mathematical formulas. These findings highlight that prompt structure alone cannot compensate for architectural and scale-related constraints inherent to limited-parameter models, and underscore the importance of evaluation under controlled experimental conditions. The results indicate that chain-of-thought prompting provides marginal improvements over zero-shot prompting; however, neither technique reliably produces mathematically valid intermediate reasoning steps in this small-scale model

Index Terms—Prompt engineering, Large language models, Reasoning, Chain-of-thought, TinyLLaMA, LM Studio, Zero-shot prompting, Few-shot prompting.

I. INTRODUCTION

Reasoning is still a major hurdle for large language models (LLMs), despite their strong performance across a wide range of natural language tasks and success in standard language tasks. While they excel at pattern recognition and language generation, they often struggle with tasks that require multi-step logical inference, like mathematical problem solving or structured reasoning. A single error in the generation process tend to propagate, leading to incorrect final answers that the model cannot easily recover from [11].

Key limitation of standard prompting approaches is that they encourage models to generate direct answers without explicitly performing intermediate reasoning steps. As a result, LLMs may produce fluent but logically inconsistent outputs when faced with complex problems. This limitation highlights the need for mechanisms that guide models toward more structured and deliberate reasoning processes.

Standard prompts often cause this by pushing models to give direct answers without showing their work. Consequently, prompt engineering has emerged as an effective approach to addressing these challenges by carefully designing input prompts that influence how models generate responses. Techniques such as chain-of-thought prompting encourage LLMs to produce intermediate reasoning steps, enabling them to decompose complex problems into manageable sub-tasks [3]. Even simple modifications, such as adding reasoning cues, have been shown to significantly improve performance on arithmetic, symbolic, and commonsense reasoning tasks [5]. More advanced strategies, including zero-shot chain-of-thought and self-consistency, further enhance reasoning reliability by eliciting structured reasoning without explicit examples and reducing errors across multiple reasoning paths [4], [5].

Observations tell us that reasoning capabilities elicited through prompt engineering are closely related to model scale. Larger language models tend to benefit more from structured prompting techniques, suggesting that reasoning ability emerges as models increase in size [14]. Motivated by these findings, this paper investigates the impact of different prompt engineering strategies on LLM reasoning performance through a controlled experimental evaluation using a locally deployed small-scale language model.

The main key-sections of this paper are as follows: (1) A systematic comparison of zero-shot, few-shot, and chain-of-thought prompting techniques applied to arithmetic and mathematical word problems using a single open-source model; (2) A detailed qualitative analysis of model outputs, identifying recurring error patterns and reasoning failures; (3) An assessment of the relationship between prompt complexity and model output quality in a resource-constrained local execution environment; and

(4) Practical insights for researchers and practitioners regarding the limitations of prompt-based reasoning enhancement in small-scale LLMs.

II. LITERATURE REVIEW

Prior research on prompt engineering for large language models (LLMs) has progressed from simple instruction design to more sophisticated strategies aimed at eliciting structured reasoning. Early studies demonstrated that models do not necessarily require few-shot examples to perform a task [12]. Instead, carefully designed zero-shot prompts can guide models toward relevant regions of their latent knowledge space, enabling effective task execution through natural language constraints [1]. These findings suggest that prompting often serves as a mechanism for task specification rather than explicit learning.

Building on this foundation, chain-of-thought (COT) prompting emerged as a major advancement for improving reasoning performance [3]. COT prompting encourages models to generate intermediate reasoning steps before producing a final answer, significantly improving performance on arithmetic, commonsense, and symbolic reasoning tasks. However, the effectiveness of COT prompting is closely tied to model scale. Smaller models often generate fluent but logically inconsistent reasoning, whereas larger models exhibit substantial improvements, indicating that reasoning ability emerges with increased parameter size [14]. Subsequent work further showed that explicit reasoning examples are not always required, as simple reasoning cues can elicit structured reasoning in a zero-shot setting [5].

To address instability and hallucination in model-generated reasoning, recent studies have explored ensemble and verification-based prompting strategies. Self-consistency improves reasoning reliability by sampling multiple reasoning paths and selecting the most consistent final answer, reducing the impact of stochastic errors [4]. Verification-based approaches further enhance performance by evaluating and ranking generated solutions using auxiliary models or scoring mechanisms [2]. These methods demonstrate improved scalability and robustness compared to single-path generation strategies.

In addition to reasoning-focused prompts, instruction and role-based prompting techniques have been proposed to better constrain model behaviour [13]. Separating instructions from contextual information and clearly defining model roles has been shown to improve response accuracy and controllability [15]. Such techniques highlight the importance of prompt structure in guiding reasoning processes effectively.

Despite these advances, prompt engineering remains highly sensitive to phrasing and cannot fully compensate for fundamental limitations in model architecture or knowledge. Most existing studies focus on large-scale benchmark evaluations, leaving limited analysis of prompt strategy comparisons under controlled experimental settings. These limitations motivate the experimental investigation presented in this paper.

A. Zero-Shot Prompting

Zero-shot prompting refers to querying a language model without providing any task-specific examples. Brown et al. [12] demonstrated that large language models such as GPT-3 can perform a wide range of tasks in a zero-shot setting. However, several studies report that zero-shot prompting often struggles with multi-step reasoning problems, particularly in mathematical and logical tasks, where intermediate reasoning steps are required [1], [5]. In the zero-shot setting, the model must rely entirely on its pre-trained knowledge and the semantic cues embedded within the question itself, without any external scaffolding or worked examples.

B. Few-Shot Prompting

Few-shot prompting improves model performance by providing a small number of input-output examples within the prompt [1], [12]. Research indicates that few-shot prompting enables models to better capture task patterns and improve accuracy compared to zero-shot settings. Despite these improvements, prior work shows that few-shot prompting does not consistently resolve reasoning errors in complex, multi-step problems [3]. Although few-shot prompting improves task understanding and reduces ambiguity, its effectiveness depends on the quality and relevance of the provided examples, particularly for complex, multi-step reasoning tasks [1].

C. Chain-of-Thought (COT) Prompting

Chain-of-Thought (COT) prompting encourages language models to generate intermediate reasoning steps before producing a final answer [3]. Wei et al. [3] demonstrated that COT prompting significantly improves performance on arithmetic, symbolic, and commonsense reasoning tasks. By decomposing complex problems into smaller logical steps, COT prompting reduces

error accumulation and enhances interpretability of model outputs [3]. Furthermore, Kojima et al. [5] demonstrated that simply appending reasoning instructions such as "Let's think step by step" to a prompt can elicit structured chain-of-thought reasoning even in the absence of explicit worked examples.

III. PROBLEM STATEMENT & RESEARCH QUESTIONS

Large Language Models (LLMs) have demonstrated impressive performance across a wide range of natural language processing tasks. However, despite these advancements, LLMs often struggle with multi-step reasoning problems, particularly in domains such as mathematical word problems and logical reasoning. These tasks require the model to maintain intermediate reasoning steps and logical consistency, which is not always achieved through standard prompt formulations.

Recent research highlights the role of prompt engineering techniques—such as zero-shot, few-shot, and chain-of-thought prompting—in improving model reasoning performance without modifying the underlying model architecture. While prior studies report that chain-of-thought prompting can significantly enhance reasoning accuracy, most existing work focuses on large-scale benchmark evaluations or proprietary, large-parameter models. Consequently, there is limited understanding of how different prompting strategies compare under controlled experimental conditions, especially when applied to open-source language models.

Furthermore, many studies emphasize quantitative benchmark scores while providing limited qualitative analysis of how prompting techniques influence reasoning behavior and interpretability of model outputs. This lack of systematic comparison makes it difficult to assess the practical effectiveness of prompt engineering techniques for smaller-scale or academic settings.

To address these gaps, this study systematically investigates the impact of zero-shot, few-shot, and chain-of-thought prompting on reasoning-based mathematical tasks using an open-source language model. By evaluating both performance accuracy and qualitative reasoning behavior, this work aims to provide clearer insights into the strengths and limitations of different prompting strategies in enhancing LLM reasoning capabilities.

A. Research Questions

- RQ1: How does zero-shot prompting perform on multi-step mathematical reasoning tasks compared to few-shot and chain-of-thought prompting?
- RQ2: Does chain-of-thought prompting provide consistent performance improvements across reasoning tasks of varying complexity?
- RQ3: How do different prompting strategies affect the interpretability and transparency of model outputs?
- RQ4: What recurring error patterns emerge when prompt engineering strategies are applied to small-scale language models?

IV. METHODOLOGY

This section describes the experimental setup used to evaluate the impact of different prompting strategies on LLM reasoning performance. The methodology was designed to facilitate a structured and replicable comparison of prompting techniques under controlled conditions, ensuring that observed differences in output quality could be attributed to prompt design rather than environmental variability.

A. Research Design

This study follows an experimental research design to evaluate the performance of a locally deployed Large Language Model (LLM) on basic arithmetic and mathematical word problems using different prompting techniques. The objective was to observe how variations in prompt structure influence the correctness and reasoning ability of the model. A comparative experimental framework was adopted to evaluate the relative effectiveness of different prompting strategies. Each problem instance was tested sequentially under zero-shot, few-shot, and chain-of-thought conditions. The responses were recorded and compared to identify performance trends and consistency patterns across strategies. This comparative design enables structured analysis rather than isolated observation.

While the study focuses on arithmetic and mathematical reasoning tasks, the findings may not directly generalize to other domains such as creative writing or domain-specific knowledge tasks. The scope is intentionally limited to controlled problem-solving scenarios to enable clear evaluation of reasoning performance. The experiment was conducted in an offline environment

using a locally installed LLM. Multiple mathematical problems were tested under controlled conditions, and the generated outputs were recorded for analysis.

B. Model Configuration

The experiment was conducted using the open-source language model TinyLLaMA-1.1B-Chat-v1.0 [9], executed through LM Studio [10] in an offline setup. The model was selected due to its lightweight architecture and compatibility with limited hardware resources, making it suitable for academic research environments without access to high-performance computing infrastructure.

TABLE I. MODEL AND SYSTEM CONFIGURATION

Parameter	Specification
Model Name	tinyllama-1.1b-chat-v1.0
Model Size	1.1 Billion Parameters
Platform Used	LM Studio
Execution Mode	Local (Offline)
Operating System	Windows

C. Prompting Techniques

Three prompting strategies were evaluated in this study:

- Zero-Shot Prompting: The model was directly given a mathematical problem without any example or additional guidance. The model was expected to rely solely on its pre-trained knowledge to formulate a response.
- Few-Shot Prompting: An example problem along with its complete solution was provided before presenting the target question. This gave the model a contextual reference for the expected output format and solution approach.
- Chain-of-Thought (COT) Prompting: The model was explicitly instructed to solve the problem step-by-step and explain its reasoning before presenting the final answer. This technique aims to elicit a structured, intermediate reasoning process.

D. Dataset Preparation

The dataset consisted of manually created mathematical problems categorized into different types to cover a range of reasoning complexities:

- Basic arithmetic operations
- Mixed arithmetic expressions involving operator precedence
- Proportional reasoning problems (unitary method)
- Speed-distance-time word problems

TABLE II. PROMPTING STRATEGY OVERVIEW

Prompting Technique	Description	Instruction Pattern Used
Zero-Shot	Direct question without example	"What is $25 \times 4 + 10$?"
Few-Shot	Question with a solved example	"Example: $5 \times 2 + 3 = 13$. Now solve..."
Chain-of-Thought	Step-by-step reasoning requested	"Solve step by step and explain clearly..."

TABLE III. PROBLEM CATEGORIES USED IN EVALUATION

Problem Type	Example Question
Basic Addition	$23 + 47$

Arithmetic Expression	$25 \times 4 + 10$
Proportion Problem	If 3 pens cost ₹45, how much do 7 pens cost?
Speed-Distance Problem	A train travels 60 km in 2 hours. How far in 5 hours?

V. EXPERIMENTAL ENVIRONMENT

This section describes the hardware and software environment used to conduct the experiments. All experiments were performed in a fully offline, locally controlled setting to ensure reproducibility and eliminate variability arising from API latency, network-dependent rate limiting, or cloud-side model versioning.

A. Software Environment

LM Studio [10] was used as the primary interface for loading and querying the language model. LM Studio is an open-source desktop application that provides a graphical interface for discovering, downloading, and running large language models locally. It supports a range of quantized GGUF model formats and provides an integrated chat interface with configurable generation parameters, including temperature, top-p sampling, and maximum token length.

The TinyLLaMA-1.1B-Chat-v1.0 model [9] was loaded via LM Studio and executed using default inference settings to maintain consistency across all experimental runs. Prompts were entered manually through the LM Studio chat interface, and all model outputs were recorded verbatim for subsequent analysis. No external APIs, fine-tuning pipelines, or retrieval-augmented generation components were employed.

B. Hardware Configuration

The experiments were conducted on a personal computing device running the Windows operating system. Given that TinyLLaMA-1.1B is a highly compressed, quantized model, no dedicated GPU was required for inference. All computations were performed using the CPU, which is consistent with the model's design objective of enabling LLM deployment in resource-constrained academic and personal environments.

C. Model Characteristics

TinyLLaMA-1.1B-Chat-v1.0 is a compact transformer-based language model trained on a large text corpus and subsequently fine-tuned for conversational instruction-following tasks. With approximately 1.1 billion parameters, it represents a small-scale LLM intended for deployment on edge devices and low-resource environments. The model employs a decoder-only transformer architecture and produces autoregressive text outputs conditioned on an input prompt sequence. Its reduced parameter count enables fast local inference but simultaneously constrains the depth of representational capacity available for complex reasoning tasks.

D. Execution Protocol

Each problem was submitted to the model three times—once under each prompting condition: zero-shot, few-shot, and chain-of-thought. Between runs, the conversation history was cleared to prevent any carry-over effects from prior interactions. All generation parameters were held constant across conditions to isolate the effect of prompt design. The recorded outputs were subsequently evaluated against mathematically correct reference solutions and assessed qualitatively for reasoning coherence and intermediate step validity.

VI. PROMPT DESIGN AND TEMPLATE STRUCTURE

This section describes the structure and rationale of each prompting technique employed in the study. The design of prompt templates was informed by established practices in the prompt engineering literature and adapted to the arithmetic and mathematical reasoning tasks under investigation.

A. Zero-Shot Prompt Templates

Zero-shot prompts were formulated as direct mathematical questions without any contextual examples or instructional cues regarding solution procedure. The model was expected to derive both the solution method and the final answer from its pre-

trained knowledge alone. This condition represents the baseline performance level and is commonly used in LLM evaluations as the default reference point.

For instance, the zero-shot prompt for the arithmetic expression task was presented as: "What is $25 \times 4 + 10$?" without any additional context. Similarly, word problems were stated in their standard natural language form without structural guidance or decomposition cues.

B. Few-Shot Prompt Templates

Few-shot prompts included one solved example immediately preceding the target problem. The solved example was selected to be structurally analogous to the target question, providing the model with a worked instance of the solution approach. This design was intended to activate relevant reasoning patterns through in-context learning rather than explicit instruction.

For the arithmetic expression task, the few-shot prompt introduced the example " $5 \times 2 + 3 = 13$ " before asking the model to solve " $25 \times 4 + 10$ ". For the proportional reasoning task, an example involving the cost of apples was provided before the analogous pen cost problem. The inclusion of a single example was intentional, reflecting the minimal few-shot setting most commonly evaluated in the literature.

C. Chain-of-Thought Prompt Templates

Chain-of-thought prompts explicitly instructed the model to reason step-by-step and articulate intermediate calculations before presenting a final answer. This technique aligns with the zero-shot chain-of-thought approach described by Kojima et al. [5], in which reasoning instructions serve as a trigger for structured output without requiring worked examples.

For example, the chain-of-thought prompt for the speed-distance problem read: "Solve step by step and explain clearly: A train travels 60 km in 2 hours. First calculate the speed of the train. Then calculate how far it will travel in 5 hours at the same speed. Show all calculation steps before giving the final answer." This formulation decomposed the problem into explicit sub-tasks to guide the model's reasoning trajectory.

D. Prompt Template Summary

The following table summarizes the prompt templates used for each problem category across all three prompting conditions:

TABLE IV. PROMPT TEMPLATES BY PROBLEM TYPE AND STRATEGY

Problem	Zero-Shot	Few-Shot	Chain-of-Thought
Addition	Solve: $23 + 47$ Explain step by step.	N/A (direct question)	Solve step by step and explain clearly: What is $23 + 47$?
Arithmetic Expression	What is $25 \times 4 + 10$?	Example: $5 \times 2 + 3 = 13$. Now solve: $25 \times 4 + 10$	Solve step by step and explain clearly: What is $25 \times 4 + 10$?
Proportion	If 3 pens cost ₹45, how much do 7 pens cost?	Example: If 2 apples cost ₹20, how much do 5 apples cost? [Solution shown]. Now solve: 3 pens...	Solve step by step: If 3 pens cost ₹45, how much do 7 pens cost? Show each calculation step.
Speed-Distance	A train travels 60 km in 2 hours. How far in 5 hours?	Example: A car travels 100 km in 4 hours... [Solution shown]. Now solve: A train...	Solve step by step: A train travels 60 km in 2 hours. First calculate speed. Then calculate distance in 5 hours.

VII. EVALUATION METRICS

This section defines the evaluation criteria applied to assess model performance across the four problem categories and three prompting conditions. Given the nature of the tasks—arithmetic computation and mathematical word problem solving—the evaluation framework was designed to capture both quantitative correctness and qualitative reasoning quality.

A. Answer Correctness

Answer correctness was determined by comparing the model's final stated answer against the mathematically verified ground truth solution. For each problem, the correct answer was computed independently and used as the reference. A response was

classified as Correct if the final answer matched the ground truth exactly or within acceptable rounding, Partially Correct if the final answer was incorrect but the model demonstrated some relevant intermediate reasoning, and Incorrect if the final answer was wrong with no evidence of valid reasoning.

This three-tier classification was adopted in preference to a binary correct-incorrect scheme in order to capture edge cases in which the model produced a wrong answer through a partially valid reasoning path, which is a meaningful distinction for understanding model behaviour.

B. Logical Consistency

Logical consistency refers to the degree to which the model's intermediate reasoning steps are mathematically coherent and correctly grounded in established arithmetic or algebraic rules. A response was classified as Consistent if all intermediate steps followed logically from the prior step and led to the stated answer, Partially Consistent if some steps were valid but others introduced errors or unsupported transformations, and Inconsistent if the intermediate steps contained fundamental mathematical errors regardless of the final answer.

This metric is particularly relevant in the chain-of-thought condition, where the model is explicitly required to produce intermediate steps. It provides a means of distinguishing between lucky correct answers achieved through invalid reasoning and genuinely correct solutions derived from sound step-by-step logic.

C. Reasoning Clarity

Reasoning clarity assesses how clearly and coherently the model's explanation communicates the solution approach to a human reader. Responses with clear, structured, and mathematically appropriate explanations were rated as high clarity. Responses that were verbose but mathematically confused, or that included irrelevant technical terminology (such as floating-point conversion procedures) unrelated to the problem, were rated as low clarity. This metric captures the degree of alignment between explanation quality and solution correctness, which is distinct from logical consistency.

D. Evaluation Summary Table

TABLE V. EVALUATION METRIC DEFINITIONS

Metric	Definition	Scale
Answer Correctness	Final answer matches the ground truth	Correct / Partially Correct / Incorrect
Logical Consistency	Intermediate steps follow valid mathematical rules	Consistent / Partially Consistent / Inconsistent
Reasoning Clarity	Explanation clearly and appropriately communicates the solution process	High / Medium / Low

VIII. RESULTS

The experimental evaluation was conducted using the TinyLLaMA-1.1B-Chat-v1.0 model on a set of arithmetic and reasoning-based problems. The tasks included basic addition, arithmetic expressions involving operator precedence, proportional reasoning problems, and speed-distance calculations. All prompts were executed under identical model settings to maintain consistency. Results were assessed across all three prompting conditions using the evaluation metrics defined in Section VII.

TABLE VI. MODEL PERFORMANCE ON ARITHMETIC AND REASONING TASKS

Problem Type	Prompting Strategy	Answer Correctness	Logical Consistency	Reasoning Clarity	Final Status
Basic Addition (23 + 47)	Zero-Shot	Correct	Partially Consistent	Low	Correct
Basic Addition (23 + 47)	Few-Shot	Correct	Partially Consistent	Low	Correct
Basic Addition (23 + 47)	CoT	Correct	Partially Consistent	Medium	Correct

Arithmetic Expression ($25 \times 4 + 10$)	Zero-Shot	Inconsistent	Inconsistent	Low	Partially Correct
Arithmetic Expression ($25 \times 4 + 10$)	Few-Shot	Partially Correct	Inconsistent	Low	Partially Correct
Arithmetic Expression ($25 \times 4 + 10$)	CoT	Correct (final)	Partially Consistent	Medium	Correct
Proportion Problem (Pens)	Zero-Shot	Incorrect	Inconsistent	Low	Incorrect
Proportion Problem (Pens)	Few-Shot	Incorrect	Inconsistent	Low	Incorrect
Proportion Problem (Pens)	CoT	Incorrect	Inconsistent	Low	Incorrect
Speed-Distance Problem	Zero-Shot	Incorrect	Inconsistent	Low	Incorrect
Speed-Distance Problem	Few-Shot	Incorrect	Inconsistent	Low	Incorrect
Speed-Distance Problem	CoT	Incorrect	Inconsistent	Low	Incorrect

A. Performance on Basic Arithmetic

For the addition problem ($23 + 47$), the model generated the correct final answer of 70 across all three prompting conditions. However, in both zero-shot and few-shot conditions, the explanation accompanying the correct answer included unnecessary references to floating-point conversion and Python's internal arithmetic mechanisms. Steps such as converting 23 to 23.0000 and 47 to 47.0000 before adding were described as essential to the computation, despite being mathematically irrelevant. The chain-of-thought prompt produced a marginally more structured explanation but retained the same erroneous floating-point framing.

For the expression $25 \times 4 + 10$, inconsistent behaviour was observed across the three prompting conditions. In the zero-shot condition, the model produced conflicting final answers across the reasoning trace, arriving at incorrect values such as 60 or 80 despite the correct answer being 110. In the few-shot condition, the presence of the example ($5 \times 2 + 3 = 13$) appeared to distort the model's computation, leading it to partially apply the example's arithmetic structure to the new problem. Only in the chain-of-thought condition did the model produce the correct intermediate step ($25 \times 4 = 100$) and final answer (85 in one instance, 110 partially approached in another), though even here the intermediate reasoning occasionally included irrelevant operations.

B. Performance on Proportional Reasoning

For the proportional reasoning task involving the cost calculation of pens, the model failed to correctly apply the unitary method under all three prompting conditions. The expected solution procedure—identifying the cost per unit by dividing the total cost by the number of units, then scaling to the target quantity—was not produced in any of the recorded outputs.

In the zero-shot condition, the model generated an unrelated arithmetic expression involving multiple multiplications and additions without any connection to the problem statement. In the few-shot condition, despite being provided with a worked example structurally identical to the target problem (the apple cost example), the model reproduced the same irrelevant arithmetic pattern rather than adapting the example's solution approach. In the chain-of-thought condition, where the model was explicitly instructed to solve step-by-step, the reasoning steps generated were similarly incoherent. The final answer consistently deviated significantly from the correct value of ₹105.

C. Performance on Speed-Distance Problems

In the motion-based problem requiring the calculation of speed followed by distance estimation, the model did not correctly apply the standard formula $\text{Speed} = \text{Distance} \div \text{Time}$ under any prompting condition. In the zero-shot condition, the model introduced unnecessary unit transformations, converting kilometres to meters and hours to seconds before performing calculations that did not correspond to any standard physics formula. The final output of 5467.28 km was significantly incorrect.

In the few-shot condition, the worked example (a car travelling 100 km in 4 hours) was provided to anchor the solution approach. However, the model did not extract the speed formula from the example and instead repeated the same erroneous unit-conversion procedure. In the chain-of-thought condition, where sub-questions explicitly instructed the model to first calculate speed and then distance, the model acknowledged the sub-task structure but failed to execute either calculation.

correctly. The step-by-step instructions did not prevent the propagation of incorrect unit conversions and formula misapplications.

D. Overall Performance Summary

Across all evaluated tasks, the following behavioural patterns were observed consistently across all three prompting strategies:

- Correct handling of the simplest arithmetic operation (addition) under all conditions.
- Inconsistent execution of expressions requiring operator precedence, with occasional correct final answers achieved through flawed intermediate reasoning.
- Complete failure in proportional reasoning tasks, with the unitary method not applied in any condition.
- Significant inaccuracies in multi-step motion-based word problems, with formula misapplication observed universally.
- Frequent and irrelevant inclusion of floating-point conversion steps not required by any of the problem types.

Overall, the model demonstrated limited reliability in structured reasoning tasks despite producing verbose explanations. Correct numerical outcomes were occasionally achieved; however, logical consistency across problem types remained unstable regardless of the prompting strategy employed.

IX. ERROR ANALYSIS

This section presents a systematic analysis of the reasoning errors observed in the model outputs across all prompting conditions and problem categories. Rather than reporting raw model responses, this analysis identifies recurring error patterns and their implications for understanding the reasoning limitations of small-scale language models under prompt engineering conditions.

A. Unnecessary Floating-Point Conversion

One of the most consistently observed error patterns across all prompting conditions was the model's tendency to introduce floating-point conversion steps that were entirely unnecessary for the problems at hand. In multiple outputs, the model described a procedure of converting integer operands into floating-point equivalents (e.g., converting 23 to 23.0000 or 47 to 47.0000) before performing basic addition. These steps were framed as essential components of the solution process, suggesting that the model had learned to associate mathematical explanation templates with floating-point conversion from its pre-training data, possibly from Python programming tutorials or numerical computing documentation.

This pattern persisted even when the chain-of-thought prompting condition explicitly requested clear, step-by-step reasoning. The model did not distinguish between the requirements of integer arithmetic and those of floating-point numerical computation, indicating an inability to adapt explanation templates to the specific constraints of the problem context.

B. Incorrect Arithmetic Transformations

For the arithmetic expression $25 \times 4 + 10$, the model exhibited a range of incorrect arithmetic transformations across different runs and prompting conditions. In several instances, the model produced intermediate steps that did not follow from the prior step, such as asserting that $25 \times 4 = 75$ (incorrect) or computing $13 \times 4 = 60$ by inappropriately incorporating the few-shot example value of 13 into the new problem's computation. These transformations suggest that the model was interpolating between memorized arithmetic patterns rather than executing a fresh computation based on the given operands.

Operator precedence handling was similarly unreliable. The standard mathematical convention of performing multiplication before addition was not consistently applied. In some outputs, the model added 4 and 10 before multiplying by 25, resulting in incorrect final answers. In others, the correct precedence was partially respected but combined with an erroneous intermediate product, yielding a partially correct but still wrong result.

C. Failure in Proportional Reasoning

The proportional reasoning task exposed a fundamental inability to apply the unitary method, which is a basic mathematical procedure taught at the primary school level. The model's outputs for this task revealed that it could not reliably decompose the problem into its constituent steps: (1) determine the cost per unit, (2) multiply by the target quantity. Instead, across zero-shot, few-shot, and chain-of-thought conditions, the model generated multi-operand arithmetic expressions involving numbers that bore no interpretable relationship to the given quantities (cost of pens or number of pens).

Notably, the few-shot condition—which provided a worked example with an identical solution structure—did not produce any measurable improvement. The model failed to transfer the solution pattern from the apple cost example to the pen cost problem, suggesting that in-context learning was ineffective for this class of reasoning in the 1.1B parameter regime.

D. Inconsistent Intermediate Steps

A recurring pattern observed across all problem types was the inconsistency between intermediate reasoning steps within a single response. In several outputs, an intermediate step would assert a value (e.g., 85 as the product of a computation) and a subsequent step would use a different value (e.g., 60) without explanation or acknowledgment of the change. This within-response inconsistency indicates that the model was generating text in a largely unconstrained autoregressive manner, without maintaining a coherent internal state that tracked the evolving values of mathematical variables across steps.

This behaviour is particularly problematic for chain-of-thought prompting, which relies on the model maintaining logical continuity across a sequence of intermediate steps. When intermediate values are inconsistent, the step-by-step format provides the appearance of structured reasoning without the substance of genuine mathematical inference.

E. Misapplication of Physical Formulas

For the speed-distance problem, the model consistently failed to apply the formula $\text{Speed} = \text{Distance} \div \text{Time}$. Instead, outputs across all prompting conditions introduced unit conversion procedures (e.g., converting kilometres to metres, hours to seconds) before performing arithmetic operations that did not correspond to any standard physics or mathematics formula. The final results were orders of magnitude removed from the correct answer (150 km), with outputs such as 5467.28 km being reported as the solution.

The chain-of-thought condition, which decomposed the problem into the sub-tasks of calculating speed and then distance, did not prevent this error. The model acknowledged the sub-task structure in its output preamble but then failed to execute either sub-task using the correct formula. This suggests that structural compliance with a prompt's format does not guarantee semantic alignment with the intended mathematical procedure.

F. Verbosity Without Accuracy

A cross-cutting observation from the error analysis is the consistent mismatch between explanation length and reasoning correctness. In nearly every evaluated output, the model produced lengthy, structured-sounding explanations with numbered steps, technical terminology, and code-like notation. However, the length and formatting of these explanations showed no positive correlation with the correctness or logical consistency of the reasoning they described.

This verbosity-accuracy dissociation is consistent with the known tendency of language models to produce fluent, well-formatted text that superficially resembles correct reasoning without the underlying computational validity. It underscores the importance of evaluating reasoning-based tasks on the basis of mathematical correctness and logical coherence rather than surface-level explanation quality.

X. DISCUSSION

The experimental findings reveal clear distinctions between the model's performance on basic arithmetic operations and multi-step reasoning tasks. While the model demonstrated correctness in simple addition, its performance declined significantly as task complexity increased. This suggests that surface-level numerical computation is handled more reliably than structured reasoning that requires logical sequencing, formula application, and contextual interpretation.

A. Arithmetic Stability

The correct output observed in the basic addition task indicates that the model retains adequate capability for straightforward numerical operations at the simplest level of arithmetic. However, even in this case, the reasoning explanation contained unnecessary references to floating-point conversions and internal Python arithmetic processes. This pattern suggests that the model relies heavily on learned textual templates rather than generating reasoning grounded in mathematical necessity. The explanation quality did not degrade for addition, but neither did it demonstrate genuine mathematical understanding; correctness appeared to arise from pattern matching rather than deliberate computation.

B. Multi-Step Reasoning Limitations

In tasks involving operator precedence, proportional reasoning, and motion-based calculations, the model exhibited logical instability. Although responses were verbose and syntactically coherent, intermediate computational steps were often inconsistent with established mathematical rules. In some instances, incorrect intermediate values still led to partially correct final answers, indicating weak internal reasoning alignment. The proportional reasoning task particularly highlighted the model's difficulty in applying the unitary method. Similarly, in the speed-distance problem, the standard formula was not applied, and unnecessary unit transformations were introduced.

C. Verbosity Versus Logical Accuracy

An important observation from the experiments is the mismatch between explanation length and correctness. The model consistently produced detailed responses; however, increased verbosity did not correlate with improved numerical accuracy. This indicates that fluent language generation does not necessarily imply reliable reasoning capability. The error analysis in Section IX reinforces this finding, revealing that well-formatted, step-numbered responses frequently contained fundamental mathematical errors that invalidated the entire reasoning chain.

D. Model Scale Considerations

The limitations observed may be partially attributed to the relatively small parameter size (1.1B) [9]. Smaller language models typically demonstrate constrained reasoning depth when compared to larger transformer-based architectures [14]. The instability in multi-step problem solving suggests restricted internal representation of mathematical structures. The results obtained in this study are consistent with the scaling hypothesis articulated in prior literature: reasoning capabilities of the kind assessed here appear to require a minimum threshold of model capacity to emerge reliably.

E. Effectiveness of Prompting Strategies

The comparison across prompting conditions reveals that none of the three strategies—zero-shot, few-shot, or chain-of-thought—consistently produced correct and logically valid outputs across all problem categories. Chain-of-thought prompting provided marginal structural improvements in some cases, particularly for the arithmetic expression problem, where the step-by-step instruction partially guided the model toward a more correct intermediate calculation. However, for proportional reasoning and speed-distance problems, the chain-of-thought format provided no measurable benefit, with the model generating structurally formatted but mathematically invalid responses.

Few-shot prompting similarly failed to produce reliable gains. In the proportional reasoning task, despite the provision of a worked example with an identical solution structure, the model did not transfer the solution approach to the target problem. This suggests that in-context learning may require significantly more model capacity than the 1.1B parameter regime to enable reliable analogical reasoning.

F. Research Implications

These results emphasize the need to distinguish between linguistic fluency and computational reasoning accuracy when evaluating language models. A model may produce grammatically coherent and technically styled explanations while failing to maintain mathematical correctness. Therefore, evaluation of reasoning-based tasks should prioritize logical consistency and numerical validation rather than explanation completeness alone. The findings also highlight the practical importance of selecting appropriately scaled models for reasoning-intensive applications.

G. Comparison with Prior Work

The experimental findings of this study contrast with several observations reported in prior prompt engineering research. While earlier studies suggest that carefully designed zero-shot prompts can effectively activate latent task-relevant knowledge [12], the evaluated small-scale model did not demonstrate stable reasoning performance under structured prompting. Chain-of-thought prompting has been widely reported to enhance arithmetic and symbolic reasoning performance, particularly in large-scale models [3]. However, the results obtained in this study indicate that such improvements are not uniformly transferable to smaller parameter models, aligning with scaling-based findings in prior literature [14]. Ensemble-based and verification-oriented strategies [2], [4] were not evaluated in the current study, representing a direction for future investigation.

XI. CONCLUSION

This paper presented a controlled experimental evaluation of prompt-based reasoning performance using the TinyLLaMA-1.1B-Chat model executed locally via LM Studio. The model was assessed on arithmetic expressions, proportional reasoning problems, and motion-based calculations under three structured prompting conditions: zero-shot, few-shot, and chain-of-thought. The evaluation focused on numerical correctness, logical consistency, and reasoning clarity across all conditions.

The results indicate that while the model performs reliably on the simplest arithmetic tasks, its reasoning capability degrades significantly in multi-step and context-dependent problems. In several instances, responses were linguistically fluent yet mathematically inconsistent. The presence of verbose, well-formatted explanations did not correspond to improved numerical accuracy, highlighting a divergence between language generation quality and computational reasoning reliability.

Contrary to prior research suggesting that structured prompting techniques such as chain-of-thought reasoning substantially enhance problem-solving performance, the findings of this study demonstrate that such improvements are not uniformly observed in smaller-scale models. The limitations identified suggest that prompt engineering alone cannot fully mitigate architectural and scale-related constraints. Reasoning robustness appears closely linked to model capacity and internal representation depth.

The error analysis revealed four primary failure modes: unnecessary floating-point conversion, incorrect arithmetic transformations, failure to apply standard mathematical formulas, and inconsistent intermediate steps that undermined logical continuity. These patterns collectively indicate that the model generates reasoning-like text through template matching rather than genuine mathematical inference, a distinction with important practical implications for the deployment of small-scale LLMs in educational or technical contexts.

The study emphasizes the importance of task-specific and controlled evaluations when assessing reasoning capabilities of language models. Reliable deployment in domains requiring mathematical precision demands not only structured prompting but also validation mechanisms or larger model architectures capable of sustained multi-step logical reasoning.

XII. FUTURE WORK

The present evaluation focused on a limited set of arithmetic and reasoning tasks using a single small-scale language model. Although the findings highlight clear reasoning limitations, the scope of the study can be expanded in several meaningful directions.

One important extension would be to conduct the same controlled experiments on larger language models under identical prompting conditions. Since prior research suggests that reasoning ability improves with scale [14], a structured comparison across different parameter sizes—such as 7B, 13B, and 70B models—would help determine whether the instability observed in this study is primarily a consequence of limited model capacity.

Another potential direction involves incorporating multi-response generation strategies such as self-consistency [4]. Instead of relying on a single reasoning path, future experiments may sample multiple outputs for each problem and analyze consistency across generated solutions to determine whether incorrect answers result from random variation or from systematic reasoning gaps.

The dataset used in this study was intentionally controlled and small in size to allow detailed analysis of reasoning behaviour. Future work may expand the evaluation to include a broader range of problem types, such as algebraic manipulation, logical deduction, and multi-variable word problems. A larger and more diverse task set would improve the generalizability of the findings and enable statistically robust performance comparisons.

Additionally, integrating external verification mechanisms may improve reliability. Combining language models with symbolic solvers or rule-based validation systems could address the computational inconsistencies identified in this evaluation. Exploring hybrid reasoning frameworks may therefore represent a practical approach to enhancing numerical accuracy without requiring larger model architectures. Finally, a systematic analysis of prompt robustness to minor phrasing variations would contribute to a deeper understanding of how prompt design interacts with model limitations in small-parameter regimes.

. REFERENCES

- [1] S. Reynolds and K. McDonell, "Prompt programming for large language models: Beyond the few-shot paradigm," arXiv preprint arXiv:2102.07350, 2021.
- [2] C. Cobbe et al., "Training verifiers to solve math word problems," arXiv preprint arXiv:2110.14168, 2021.

- [3] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," arXiv preprint arXiv:2201.11903, 2022.
- [4] X. Wang et al., "Self-consistency improves chain of thought reasoning in language models," arXiv preprint arXiv:2203.11171, 2022.
- [5] T. Kojima et al., "Large language models are zero-shot reasoners," arXiv preprint arXiv:2205.11916, 2022.
- [6] A. H. K. et al., "Interpretable models capable of handling systematic missingness in imbalanced classes and heterogeneous datasets," arXiv preprint arXiv:2206.02056, 2022.
- [7] DAIR.AI, "Prompt engineering guide," 2023.
- [8] OpenAI, "Prompt engineering," OpenAI API Documentation, 2023.
- [9] TinyLLaMA Contributors, "TinyLLaMA: An open-source small language model," 2023.
- [10] LM Studio, "LM Studio: Discover, download, and run local LLMs," 2024.
- [11] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer, "Scheduled sampling for sequence prediction with recurrent neural networks," in Proc. Adv. Neural Inf. Process. Syst. (NIPS), 2015, pp. 1171-1179.
- [12] T. B. Brown et al., "Language models are few-shot learners," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2020, pp. 1877-1901.
- [13] L. Ouyang et al., "Training language models to follow instructions with human feedback," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2022, pp. 27730-27744.
- [14] J. Wei et al., "Emergent abilities of large language models," Trans. Mach. Learn. Res. (TMLR), 2022.
- [15] Y. Zhou et al., "Large language models are human-level prompt engineers," in Proc. Int. Conf. Learn. Represent. (ICLR), 2023.